

Aiken validator coverage — rejection points and mutation testing

Run: 2026-09-22T02:22:24.103Z-02:22:37.552Z · aiken v1.1.21+42babe5 · 102 tests, 102 green.

Why this is a separate report

Aiken does not measure line coverage the way the TypeScript stack does. The four TypeScript parts (`apps/api`, `apps/web`, `packages/shared`, `packages/cardano`) use `vitest`'s `v8` provider, which instruments every line, branch, statement and function and reports a percentage — that is what [test-report.pdf](#) covers. Aiken's toolchain has no equivalent: `aiken check` runs the test suite and, separately, a `--property-coverage` flag exists but does not produce a line percentage.

So the validator's coverage evidence is built with two purpose-written scripts (`contracts/scripts/rechazos-mutantes.mjs` and `contracts/scripts/rechazos-trazas.mjs`) that answer the same question — *"is every rejection path actually exercised by a test?"* — with methods that fit how a Plutus validator fails:

1. **Rejection point → test.** Every `expect` in the validator is a point where a malformed or invalid transaction aborts. For each one, the table below names the test(s) that trigger it.
2. **Mutation testing.** For every other check that rejects a transaction (an `and` condition, a boolean `expect`, a list pattern, a `when` branch, a `fail`), the script removes or weakens that one check, reruns the suite, and requires at least one test to turn red. A check that can be deleted without failing any test is a check nothing actually defends.

Both scripts run against `contracts/validators/stage.ak` and `contracts/lib/propnexus/fsm.ak` (the stage-transition validator and the shared finite-state machine it enforces), and are **not** run in CI — each mutant re-runs `aiken check`, which takes several minutes for all 51. They are run by hand and their output is what this document records.

Rejection point → test

18 `expect` statements: 18 with a test, 0 without.

Every `expect` in the validator is a point where the transaction aborts if the pattern it expects does not match. This table maps each one to the test(s) that make it abort exactly there.

File:line	expect	Tests that abort there
validators/stage.ak:69	<code>expect Finite(lower) = range.lower_bound.bound_type</code>	<code>spend_rejects_open_ended_validity_range</code> , <code>spend_rejects_missing_lower_bound</code>
validators/stage.ak:70	<code>expect Finite(upper) = range.upper_bound.bound_type</code>	<code>spend_rejects_missing_upper_bound</code>
validators/stage.ak:84	<code>expect Some(old_datum) = datum</code>	<code>spend_rejects_missing_datum</code>
validators/stage.ak:85	<code>expect Some(own_input) = find_input(tx.inputs, own_ref)</code>	<code>spend_rejects_unknown_own_ref</code>
validators/stage.ak:87	<code>expect Script(own_policy) = own_address.payment_credential</code>	<code>spend_rejects_own_input_not_locked_by_a_script</code>
validators/stage.ak:91	<code>expect assets.quantity_of(own_input.output.value, own_policy, old_datum.stage_ref,) == 1</code>	<code>spend_rejects_utxo_without_thread_token</code> , <code>spend_rejects_thread_token_of_another_stage</code> , <code>spend_rejects_utxo_with_two_units_of_own_token</code> , <code>spend_rejects_own_input_with_two_units_when_a_decoy_absorbs_carrying_thread</code>
validators/stage.ak:100	<code>expect inputs_at_address(tx.inputs, own_address) == 1</code>	<code>spend_rejects_two_script_inputs</code>
validators/stage.ak:101	<code>expect [continuing_output] = at_address(tx.outputs, own_address)</code>	<code>spend_rejects_a_second_output_at_the_exact_script_address</code> , <code>spend_rejects_two_script_outputs</code> , <code>spend_rejects_no_continuing_output</code>
validators/stage.ak:104	<code>expect [_] = carrying_thread(tx.outputs, own_policy, old_datum.stage_ref)</code>	<code>spend_rejects_thread_split_across_a_staking_variant_of_the_script_address</code> , <code>spend_rejects_value_drain</code> , <code>spend_rejects_dropping_the_thread_token</code>
validators/stage.ak:107	<code>expect list.has(tx.extra_signatories, admin)</code>	<code>spend_rejects_missing_admin_signature</code> , <code>mint_rejects_missing_admin_signature</code>
validators/stage.ak:109	<code>expect InlineDatum(raw_new_datum) = continuing_output.datum</code>	<code>spend_rejects_non_inline_datum</code>
validators/stage.ak:110	<code>expect new_datum: StageDatum = raw_new_datum</code>	<code>spend_rejects_new_datum_of_the_wrong_type</code>
validators/stage.ak:125	<code>expect continuing_output.value == own_input.output.value</code>	<code>spend_rejects_ada_drain_while_keeping_the_token</code>
validators/stage.ak:144	<code>expect list.has(tx.extra_signatories, admin)</code>	<code>spend_rejects_missing_admin_signature</code> , <code>mint_rejects_missing_admin_signature</code>
validators/stage.ak:150	<code>expect [Pair(asset_name, 1)] = dict.to_pairs(assets.tokens(tx.mint, policy_id))</code>	<code>mint_rejects_two_units_of_the_thread</code> , <code>mint_rejects_two_threads_in_one_tx</code> , <code>mint_rejects_two_asset_names_when_both_are_present_in_the_output</code> , <code>mint_rejects_a_burn</code>

File:line	expect	Tests that abort there
validators/stage.ak:153	expect [thread_output] = carrying_thread(tx.outputs, policy_id, asset_name)	mint_rejects_token_not_locked_in_the_script, mint_rejects_output_holding_extra_units_of_the_token, mint_rejects_two_outputs_each_carrying_one_unit
validators/stage.ak:154	expect InlineDatum(raw_datum) = thread_output.datum	mint_rejects_non_inline_datum
validators/stage.ak:155	expect initial_datum: StageDatum = raw_datum	mint_rejects_initial_datum_of_the_wrong_type

Mutation testing

51 mutants: 51 killed, 0 survived, 0 invalid.

Each row removes or weakens one check the validator uses to reject a transaction — an `and` condition, a boolean `expect`, a list pattern, a `when` branch, or the trailing `fail` — and reruns the suite. "Killed" means at least one test turned red without that check; a mutant that survives means nothing defends it.

File:line	What was removed	Original check	Result	Tests that caught it
validators/stage.ak:37	filter condition	<code>list.filter(outputs, fn(output) { output.address == address })</code>	killed	<code>spend_accepts_an_unrelated_output_elsewhere</code>
validators/stage.ak:51	<code>and</code> conjunct	<code>output.address.payment_credential == Script(policy),</code>	killed	<code>mint_rejects_token_not_locked_in_the_script</code>
validators/stage.ak:52	<code>and</code> conjunct	<code>assets.quantity_of(output.value, policy, stage_ref) == 1,</code>	killed	<code>mint_rejects_output_holding_extra_units_of_the_to</code>
validators/stage.ak:60	filter condition	<code> > list.filter(fn(input) { input.output.address == address })</code>	killed	<code>spend_accepts_an_extra_wallet_input</code>
validators/stage.ak:69	accepted an open lower bound	<code>expect Finite(lower) = range.lower_bound.bound_type</code>	killed	<code>spend_rejects_missing_lower_bound</code>
validators/stage.ak:70	accepted an open upper bound	<code>expect Finite(upper) = range.upper_bound.bound_type</code>	killed	<code>spend_rejects_missing_upper_bound</code>
validators/stage.ak:72	<code>and</code> conjunct	<code>now >= lower,</code>	killed	<code>spend_rejects_timestamp_one_below_lower_bound</code>
validators/stage.ak:73	<code>and</code> conjunct	<code>now <= upper,</code>	killed	<code>spend_rejects_timestamp_outside_validity_range,</code> <code>spend_rejects_timestamp_one_above_upper_bound</code>
validators/stage.ak:91	boolean expect	<code>expect assets.quantity_of(own_input.output.value, own_policy, old_datum.stage_ref,) == 1</code>	killed	<code>spend_rejects_own_input_with_two_units_when_a_dec</code>
validators/stage.ak:100	boolean expect	<code>expect inputs_at_address(tx.inputs, own_address) == 1</code>	killed	<code>spend_rejects_two_script_inputs</code>
validators/stage.ak:101	list pattern: extra elements	<code>expect [continuing_output] = at_address(tx.outputs, own_address)</code>	killed	<code>spend_rejects_a_second_output_at_the_exact_script</code>
validators/stage.ak:104	list pattern: extra elements	<code>expect [_] = carrying_thread(tx.outputs, own_policy, old_datum.stage_ref)</code>	killed	<code>spend_rejects_thread_split_across_a_staking_varia</code>
validators/stage.ak:107	boolean expect	<code>expect list.has(tx.extra_signatories, admin)</code>	killed	<code>spend_rejects_missing_admin_signature</code>
validators/stage.ak:125	boolean expect	<code>expect continuing_output.value == own_input.output.value</code>	killed	<code>spend_rejects_ada_drain_while_keeping_the_token</code>
validators/stage.ak:136	<code>and</code> conjunct	<code>valid_datum_evolution(old_datum, new_datum, to, completion),</code>	killed	<code>spend_rejects_new_datum_of_the_wrong_type,</code> <code>spend_rejects_invalid_transition, spend_rejects_l</code>
validators/stage.ak:137	<code>and</code> conjunct	<code>time_ok,</code>	killed	<code>spend_rejects_timestamp_outside_validity_range,</code> <code>spend_rejects_open_ended_validity_range,</code> <code>spend_rejects_timestamp_one_below_lower_bound</code> and
validators/stage.ak:144	boolean expect	<code>expect list.has(tx.extra_signatories, admin)</code>	killed	<code>mint_rejects_missing_admin_signature</code>
validators/stage.ak:150	list pattern: extra elements	<code>expect [Pair(asset_name, 1)] = dict.to_pairs(assets.tokens(tx.mint, policy_id))</code>	killed	<code>mint_rejects_two_asset_names_when_both_are_presen</code>
validators/stage.ak:150	list pattern: any quantity	<code>expect [Pair(asset_name, 1)] = dict.to_pairs(assets.tokens(tx.mint, policy_id))</code>	killed	<code>mint_rejects_a_burn</code>

File:line	What was removed	Original check	Result	Tests that caught it
validators/stage.ak:153	list pattern: extra elements	expect [thread_output] = carrying_thread(tx.outputs, policy_id, asset_name)	killed	mint_rejects_two_outputs_each_carrying_one_unit
validators/stage.ak:158	and conjunct	initial_datum.stage_ref == asset_name,	killed	mint_rejects_asset_name_not_matching_stage_ref
validators/stage.ak:159	and conjunct	valid_initial_datum(initial_datum),	killed	mint_rejects_starting_outside_pending, mint_rejec
validators/stage.ak:164	fail	fail	killed	else_rejects_other_script_purposes
lib/propnexus/fsm.ak:95	when branch	Pending -> to == InProgress	killed	t_pending_to_pending, t_pending_to_observed, t_per
lib/propnexus/fsm.ak:96	when branch	InProgress -> to == Observed to == Completed	killed	t_in_progress_to_pending, t_in_progress_to_in_pro
lib/propnexus/fsm.ak:97	when branch	Observed -> to == InProgress	killed	t_observed_to_completed, t_observed_to_pending, t_
lib/propnexus/fsm.ak:98	when branch	Completed -> False	killed	t_completed_is_terminal_to_pending, t_completed_i t_completed_is_terminal_to_observed and 3 more
lib/propnexus/fsm.ak:106	and conjunct	new.project_ref == old.project_ref,	killed	t_identity_rejects_project_swap
lib/propnexus/fsm.ak:107	and conjunct	new.stage_ref == old.stage_ref,	killed	t_identity_rejects_stage_swap, spend_rejects_iden
lib/propnexus/fsm.ak:108	and conjunct	new.sequence_order == old.sequence_order,	killed	t_identity_rejects_sequence_swap
lib/propnexus/fsm.ak:109	and conjunct	new.validation_critical == old.validation_critical,	killed	t_identity_rejects_criticality_swap
lib/propnexus/fsm.ak:122	if body	bytearray.length(evidence_root) == commitment_length	killed	t_critical_needs_a_full_commitment, t_evolution_rejects_critical_without_evidence, spend_rejects_completing_critical_without_evidenc
lib/propnexus/fsm.ak:136	and conjunct	new.state == to,	killed	t_evolution_rejects_datum_state_mismatch
lib/propnexus/fsm.ak:137	and conjunct	valid_transition(old.state, to),	killed	t_evolution_rejects_leaving_terminal, spend_rejec
lib/propnexus/fsm.ak:138	and conjunct	identity_preserved(old, new),	killed	spend_rejects_identity_rewrite
lib/propnexus/fsm.ak:143	and conjunct	to == Completed,	killed	t_evolution_rejects_completion_payload_matching_a
lib/propnexus/fsm.ak:144	and conjunct	new.evidence_root == evidence_root,	killed	t_evolution_rejects_evidence_root_mismatch_betwee
lib/propnexus/fsm.ak:145	and conjunct	new.completed_at == now,	killed	t_evolution_rejects_completed_at_mismatch_between
lib/propnexus/fsm.ak:146	and conjunct	now > 0,	killed	t_evolution_rejects_zero_timestamp
lib/propnexus/fsm.ak:147	and conjunct	completion_evidence_ok(old.validation_critical, evidence_root),	killed	t_evolution_rejects_critical_without_evidence, spend_rejects_completing_critical_without_evidenc
lib/propnexus/fsm.ak:151	and conjunct	to != Completed,	killed	t_evolution_rejects_completion_without_payload
lib/propnexus/fsm.ak:152	and conjunct	new.evidence_root == old.evidence_root,	killed	t_evolution_rejects_evidence_rewrite_on_flag. prop_non_completing_evolution_preserves_evidence
lib/propnexus/fsm.ak:153	and conjunct	new.completed_at == old.completed_at,	killed	t_evolution_rejects_completed_at_rewrite_on_flag
lib/propnexus/fsm.ak:162	and conjunct	len > 0,	killed	t_initial_rejects_empty_refs
lib/propnexus/fsm.ak:163	and conjunct	len <= max_ref_length,	killed	t_initial_rejects_ref_longer_than_an_asset_name
lib/propnexus/fsm.ak:173	and conjunct	d.state == Pending,	killed	t_initial_rejects_non_pending_state, mint_rejects.
lib/propnexus/fsm.ak:174	and conjunct	d.evidence_root == "",	killed	t_initial_rejects_preloaded_evidence
lib/propnexus/fsm.ak:175	and conjunct	d.completed_at == 0,	killed	t_initial_rejects_preloaded_timestamp
lib/propnexus/fsm.ak:176	and conjunct	d.sequence_order > 0,	killed	t_initial_rejects_non_positive_sequence
lib/propnexus/fsm.ak:177	and conjunct	valid_ref(d.project_ref),	killed	t_initial_rejects_empty_refs

File:line	What was removed	Original check	Result	Tests that caught it
lib/propnexus/fsm.ak:178	and conjunct	valid_ref(d.stage_ref),	killed	t_initial_rejects_empty_refs, t_initial_rejects_ref_longer_than_an_asset_name

The 7 `expect` statements that destructure or cast a value, rather than reject on a condition, are not mutated — they are covered by the rejection-point table above instead.

How to reproduce

```
node contracts/scripts/rechazos-trazas.mjs      # rejection point → test, ~seconds
node contracts/scripts/rechazos-mutantes.mjs    # mutation testing, ~4s per mutant, several minutes total
```